# Comparing Reinforcement Learning, Monte Carlo Tree Search, and Minimax for Connect 4: A Three-Paradigm Tournament

*William Huang, Joseph Winterlich, Soham Santra*

## 1. Introduction

Artificial intelligence approaches to game-playing can be broadly classified into two categories, which are search-based agents and learning-based agents. Search-based methods work by explicitly exploring potential future game states and using the results of these explorations to inform move selection. Minimax with alpha-beta pruning, the classical approach to adversarial search, evaluates positions by recursively assuming optimal play from both sides and prunes branches that cannot influence the final decision. Monte Carlo Tree Search (MCTS) takes a different approach, rather than exhaustively evaluating positions, it repeatedly simulates games from the current state, using random or heuristic rollouts to estimate the value of each move and gradually focusing its search on the most promising branches. Learning-based agents, by contrast, acquire strategic knowledge through experience rather than explicit search. Deep reinforcement learning (RL) agents trained through self-play learn a mapping from board positions to move probabilities and position evaluations by playing thousands or millions of games against themselves, adjusting their neural network weights based on which games were won or lost. Each of these three strategies embodies a different philosophy of how an agent should reason about a game, and the best approach is not universal but depends on the structural properties of the specific game.

Connect 4 is a two-player, zero-sum, perfect-information board game played on a 7-column, 6-row vertical grid in which players alternate dropping pieces, aiming to align four consecutive pieces in any direction. Because it is both perfect-information and zero-sum, both players have complete knowledge of the board state and legal moves at all times, and any advantage gained by one agent or player comes at an exactly equal cost to the other. These properties make Connect 4 a natural environment for comparing AI agents. Connect 4 is also a solved game. Allen (1989) and Allis (1989) independently demonstrated through computational proof that the first player can force a win with perfect play from the opening position. This solved status is significant because it establishes an absolute performance ceiling against which any agent can be measured. Unlike other games with large state spaces, such as chess, for which an optimal sequence of moves is unknown, Connect 4 allows us to compare how closely each agent can approach the theoretical optimal sequence of moves.

A central finding in the game-AI literature is that the structural properties of a game, particularly its branching factor and the density of tactical forcing sequences, determine which algorithmic paradigm is best suited to play it. Connect 4 has a branching factor of only seven (Nasa et al., 2018), meaning at most seven legal moves are available from any given position. Baier and Winands (2018) identified branching factor as a primary determinant of which model dominates, games with low branching factors and high tactical density, precisely the characteristics of Connect 4, maximize Minimax's structural advantage over sampling-based methods. This suggests that classical Minimax search may outperform both MCTS and learned agents in Connect 4.

This project investigates a head-to-head comparison of three pure AI paradigms for Connect 4 which are a Minimax agent with alpha-beta pruning, a Monte Carlo Tree Search agent with heuristic rollouts, and a deep reinforcement learning agent trained exclusively through

self-play. Each agent is implemented separately with no shared components or information exchange between approaches, ensuring a fair comparison of the models themselves rather than any particular implementation detail. We evaluate all agents through a round-robin tournament, reporting first-player and second-player win rates separately to account for Connect 4's inherent first-player advantage. Based on the literature on branching factor and tactical density, we hypothesize that Minimax will outperform both the RL and MCTS agents.

## 2. Methods

All three agents were implemented in Python using PyTorch for the reinforcement learning components and NumPy for the game engine and search algorithms. The game engine supports both a standard sequential interface with make_move and undo_move operations, used by the Minimax and MCTS agents, and a vectorized batch interface operating on NumPy arrays of shape (n_envs, 6, 7), used exclusively for RL training. All experiments were conducted on an NVIDIA RTX 2080 Ti GPU with CUDA 11.8. The following subsections describe each agent's design in detail.

### *2.1 Minimax Agent*

The Minimax agent implements the classical adversarial search algorithm with alpha-beta pruning, which eliminates branches of the game tree that cannot affect the final decision under the assumption of optimal play from both sides. Alpha-beta pruning reduces the effective branching factor from b to approximately $b^{1/2}$ with perfect move ordering (Knuth & Moore, 1975), allowing the agent to search significantly deeper within the same time constraint. The agent performs a fixed-depth search to a configurable depth limit, with depth 7 used as the primary tournament configuration for testing.

Move ordering is critical to the performance of Minimax with alpha-beta pruning, as the search process is more efficient when the best move is found quickly. Our agent orders moves by proximity to the center column, which is strategically dominant in Connect 4 because it is in the most possible four-in-a-row combinations. Legal moves are sorted so that the center column is explored first, followed by adjacent columns in order moving outwards. The evaluation heuristic assigns scores to non-terminal board positions by examining each window of four consecutive cells across all four directions (horizontal, vertical, and both diagonals). Each window is scored based on the count of the agent's pieces and the opponent's pieces within it, and windows containing pieces from both players score zero, because they cannot lead directly to a win for either player. The model also awards a bonus for pieces placed in the middle column of the board. Terminal positions are scored with large constants determined by the depth remaining in the game to prefer faster wins and slower losses, meaning the agent doesn't give up if it finds itself in a bad position.

### *2.2 Monte Carlo Tree Search Agent*

The MCTS agent implements the Upper Confidence bounds applied to Trees (UCT) algorithm (Kocsis & Szepesvári, 2006), which balances exploration of less-visited moves against exploitation of moves with high estimated win rates. The selection formula uses an exploration weight of $\sqrt{2} \approx 1.414$. At each simulation, the algorithm traverses the tree from the root, expands a new leaf node, performs a rollout to a terminal state, and backpropagates the result through all visited nodes. Our agent runs 700 iterations per move in the primary tournament configuration, with additional evaluations at 200, 500, 1000, 1500, and 2000 iterations for scaling analysis.

Several enhancements improve the agent's performance beyond basic UCT. First, the agent implements tree reuse across consecutive moves, meaning that rather than discarding the search tree after each decision, the agent retains the subtree rooted at the chosen move and the opponent's response, preserving accumulated statistics from previous searches. Second, both node expansion and the rollout policy use the same tactical priority ordering: winning moves are explored first, followed by moves that block the opponent's immediate threats, then moves that do not create an immediate winning opportunity for the opponent, and finally all remaining moves, with center proximity used as a tiebreaker within each tier. During rollouts, when no tactical move is forced, the agent selects randomly from the top three safe center-ordered moves. Third, before committing any simulation budget, the agent checks whether the current position contains an immediate winning move or an immediate threat that must be blocked, and plays that move without search. This check avoids wasting simulations on positions with obvious next moves.

### *2.3 Reinforcement Learning Agent*

The reinforcement learning agent is built on a dual-headed residual convolutional neural network called the PolicyValueNet. The network takes a four-channel board encoding as input which are the current player’s pieces, the opponent’s pieces, a binary side-to-move plane, and a normalized column-height map. This representation is always computed from the current player’s perspective so the network sees a consistent view regardless of which side it is playing. The architecture consists of an initial 3×3 convolutional layer projecting to 128 feature channels, followed by six residual blocks with skip connections. The output feeds into a policy head that produces logits over the seven columns and a value head that produces a scalar win estimate in [−1, +1] via a tanh activation. The network contains approximately 2.3 million parameters.

Training is conducted exclusively through self-play, during which the agent plays games against copies of itself with no external opponents, teachers, or data from the other agents. We use 512 parallel game environments implemented in a vectorized NumPy engine (VecConnect4) to batch all forward passes through the network, achieving 93 to 165 episodes per second. Moves are selected using epsilon-greedy exploration with a softmax temperature, both decaying over the course of training. A tactical override forces immediate winning or blocking moves during self-play games, ensuring the training data does not contain trivially avoidable one-ply blunders. Experience is stored in a replay buffer of 1,000,000 transitions backed by pre-allocated NumPy arrays, and the loss combines cross-entropy on the policy head, mean squared error on the value head, and an entropy regularization term.

The final model used in the tournament is the product of three successive training runs. The first run (v1) used a rule-based opponent rather than pure self-play and was abandoned because this violated the project's requirement that each paradigm operate independently. The second run (v2) used pure self-play but contained two critical bugs discovered during training. Firstly, the replay buffer used Python's collections.deque, which has O(n) random access and caused sampling throughput to degrade as the buffer filled, and secondly, the cosine annealing learning rate scheduler was misconfigured so that the learning rate remained effectively constant throughout training. The model collapsed to near-zero policy entropy and never beat any non-random opponent. The third run (v3) fixed both bugs, added the tactical override, and trained for 500,000 episodes from scratch. This is the checkpoint used in all tournament evaluations.

### *2.4 Evaluation Protocol*

All agents are evaluated through a round-robin tournament in which each pair of agents plays a series of games, with each agent taking turns as both the first and second player. First-player and second-player win rates are reported separately, as Connect 4 is a solved first-player win and this asymmetry can significantly affect results. The primary tournament uses Minimax at depth 7, MCTS at 700 iterations, and the best RL checkpoint from the 500,000-episode training run. Additional evaluations test MCTS at iteration counts of 200, 500, 1000, 1500, and 2000 against Minimax-7, and both MCTS-700 and RL against Minimax at depths 3, 5, 7, and 9, to characterize how performance scales with computation constraint. Baseline calibration games are played against a random agent and a rule-based agent that immediately plays winning moves, blocks opponent wins, and otherwise prefers the center column. The RL agent's learning curve is tracked by evaluating checkpoints at 100,000-episode intervals against Minimax-7.

## 3. Results

### *3.1 Core Head-to-Head Results*

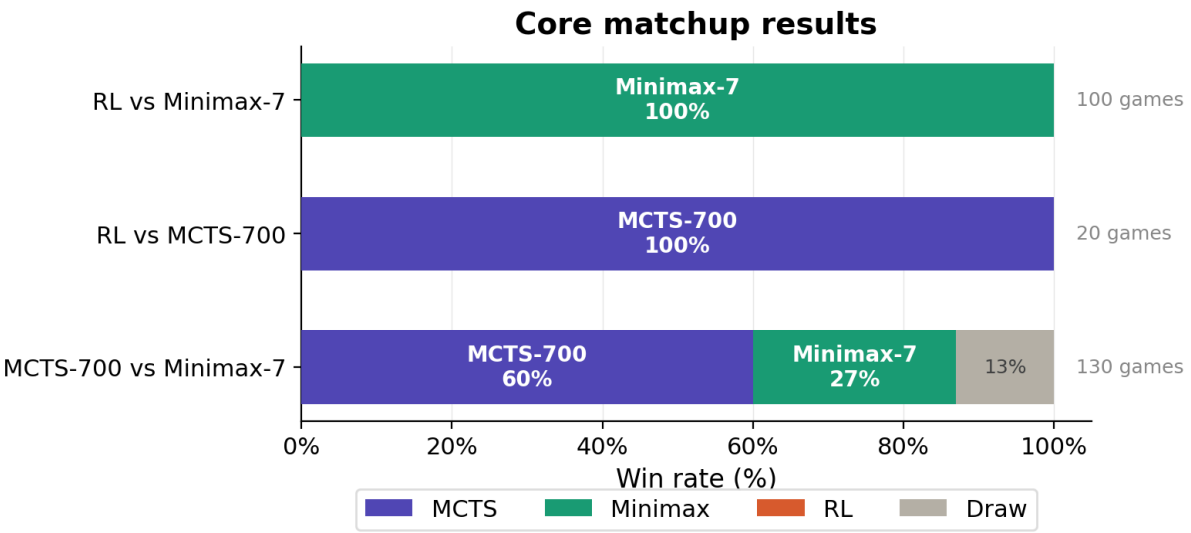


**Figure 1: Core matchup results**

The round-robin tournament produced a clear dominance hierarchy among the three agents, though not the one we initially hypothesized. The overall ranking was MCTS > Minimax > RL, with MCTS-700 emerging as the strongest agent rather than Minimax. In the core matchups, Minimax at depth 7 defeated the RL agent in all 100 games played, winning 50 as first player and 50 as second player with an average game length of only 17.6 moves, indicating that the RL agent was being tactically overwhelmed quite quickly. MCTS-700 similarly defeated the RL agent in all 20 games played, also winning from both sides. The most informative matchup was MCTS-700 versus Minimax-7, where MCTS-700 won 60% of 130 games (combining the initial 20-game, extended 50-game, and supplementary 30-game evaluation batches across multiple runs), with Minimax-7 winning 27% and 13% ending in draws. MCTS-700 won consistently from both sides, taking 14 of 25 games as first player and 16 of 25 as second player in the extended run, suggesting that its advantage was not simply down to which agent started the game (see Figure 1 above).

### *3.2 Baseline Calibration*

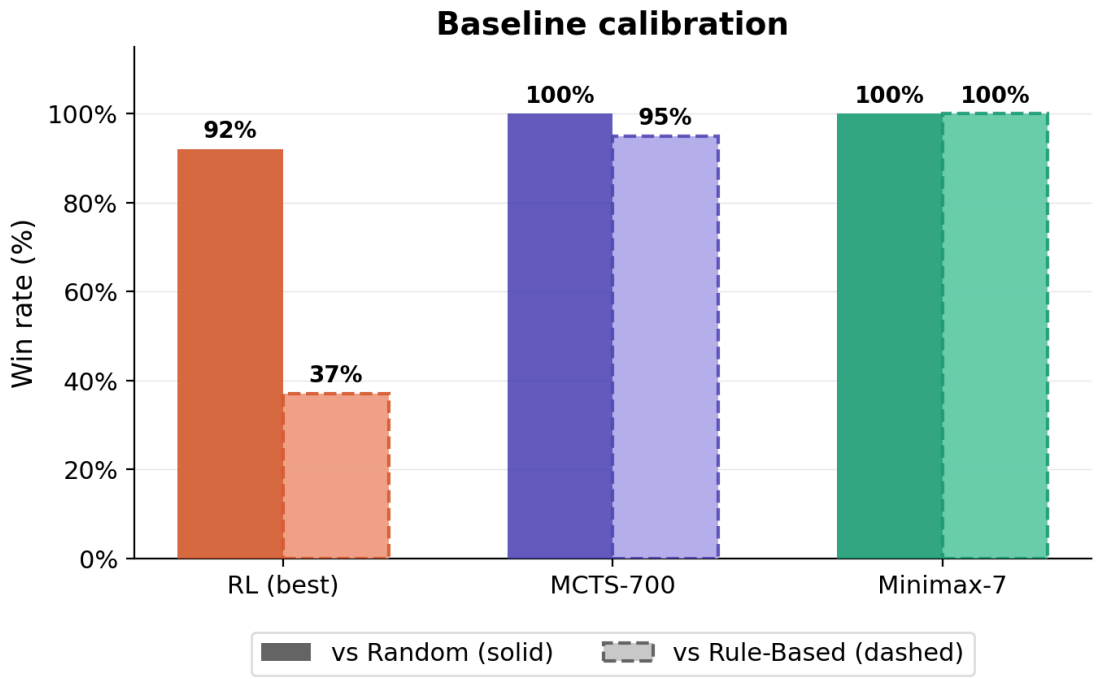


**Figure 2: Baseline calibration win rates**

To establish a performance floor, all three agents were evaluated against a random agent and a rule-based agent that plays winning moves immediately, blocks opponent wins, and otherwise prefers the center column. Minimax-7 defeated both baselines in 100% of games. MCTS-700 achieved 100% against random and 95% against rule-based over 20 games each. The RL agent achieved 92% against random over 100 games, but only 37% against rule-based over 100 games, losing 59 of those games. This baseline calibration reveals the RL agent's fundamental weakness which is despite learning to outperform random play, it had not reliably acquired even the basic win-immediately and block-immediately tactics that the simple rule-based heuristic implements. The gap between 92% versus random and 37% versus rule-based indicates that the RL agent learned some positional preferences but lacked consistent tactical awareness even at the one-move level, even though tactical overrides were used during training, as can be seen in Figure 2 above.

### *3.3 Minimax Depth Scaling*

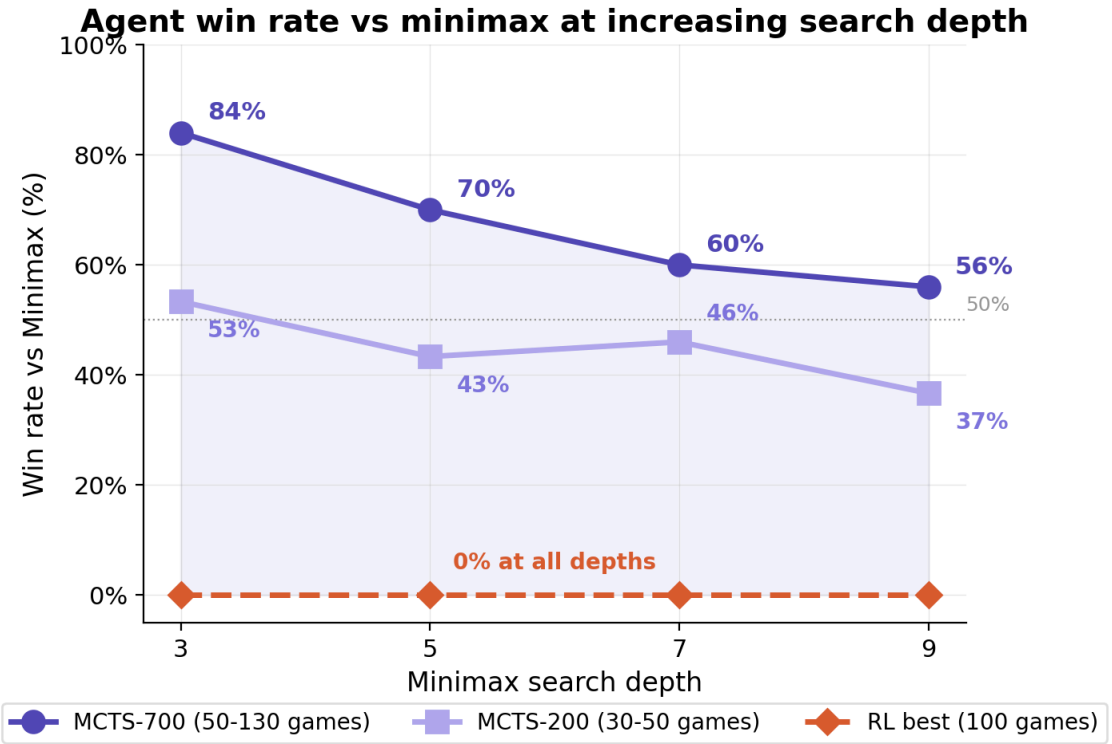


**Figure 3: Win rates of different agents against varying-depth Minimax agents**

To understand how each agent responds to increasing Minimax search depth, we evaluated both MCTS-700 and the RL agent against Minimax at depths of 3, 5, 7, and 9. The RL agent lost 100% of games at every depth tested, including depth 3, which searches only three layers ahead. This result is striking because Minimax at depth 3 uses only 0.004 seconds per move, the same decision speed as the RL agent itself, yet still defeats it perfectly. MCTS-700 showed a gradual decline in win rate as Minimax depth increased, from 84% against depth 3 to 70% against depth 5, 60% against depth 7, and 56% against depth 9. MCTS-200, tested across the same Minimax depth range, followed a similar but lower trajectory that is 53% against depth 3, 43% against depth 5, 46% against depth 7, and 37% against depth 9. Notably, MCTS-700 maintained a winning record against Minimax even at depth 9, where Minimax

searches nine layers deep and spends an average of 3.9 seconds per move compared to MCTS-700's 5.8 seconds. These results are displayed in Figure 3 above.

### *3.4 MCTS Iteration Scaling*

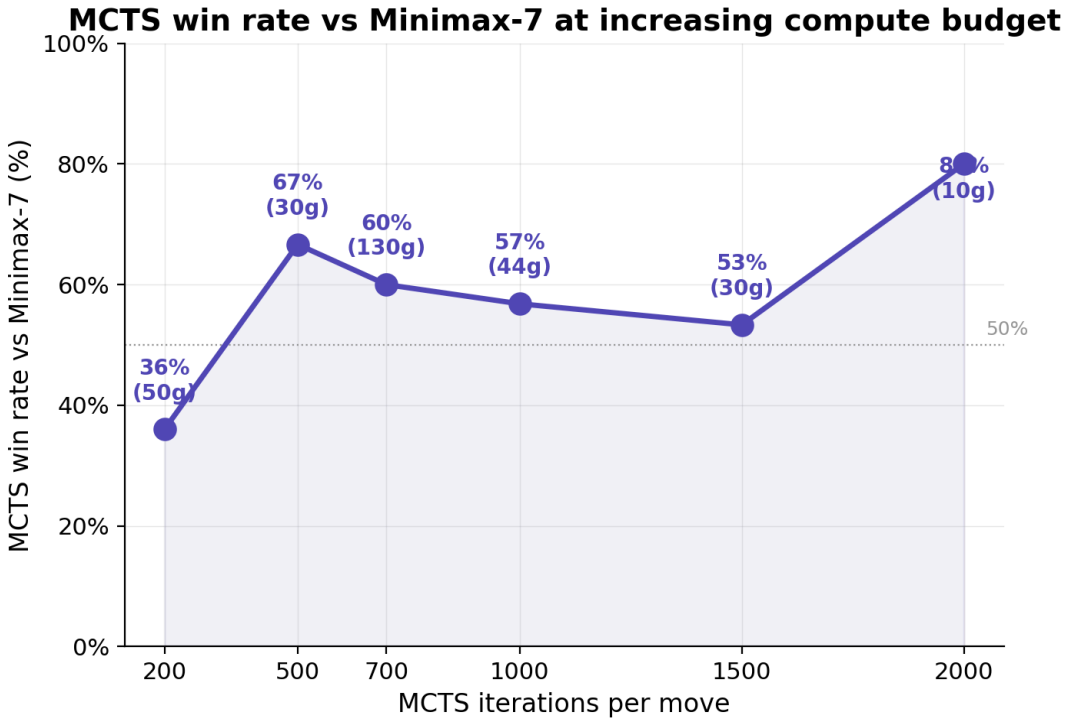


**Figure 4: MCTS win rate with increasing computation allowance**

We evaluated MCTS at iteration counts of 200, 500, 700, 1000, 1500, and 2000 against Minimax-7 to determine how MCTS performance scales with compute budget. MCTS-200 won approximately 36% of its 50 games against Minimax-7, the only configuration where MCTS had a losing record. Performance jumped sharply to 67% at 500 iterations, then stabilized in the 53–60% range for 700 through 1500 iterations, before rising again to 80% at 2000 iterations. It's worth noting, however, that this highest data point was based on only 10 games due to the length of games with the MCTS-2000 agent. The downwards trend in the 700–1500 range is likely due to sampling variance across the 30-game evaluation batches rather than true behavior, as the combined 130-game sample for MCTS-700 versus

Minimax-7 produced a stable 60% win rate. The overall trend is clear as displayed in Figure 4 above, showing MCTS at 200 iterations per move is roughly competitive with Minimax-7, and at 500 or more iterations it consistently outperforms it. The per-move time cost scales linearly with iterations, from 2.0 seconds at 200 iterations to 15.7 seconds at 2000.

### *3.5 Computation Cost Comparison*

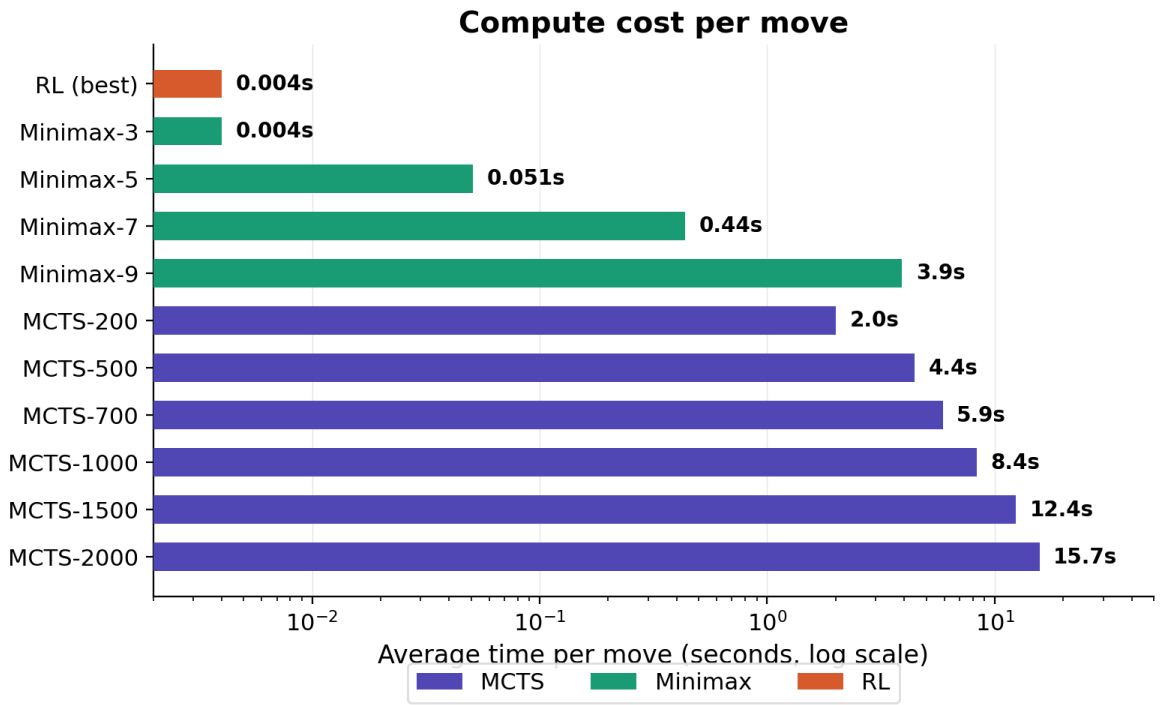


**Figure 5: Per-move compute time for each model**

The three models differ dramatically in per-move computation time. The RL agent requires only 0.004 seconds per move, consisting of a single forward pass through the neural network, making it by far the fastest agent. Minimax scales exponentially with depth, with 0.004 seconds at depth 3, 0.051 seconds at depth 5, 0.44 seconds at depth 7, and 3.9 seconds at depth 9. Meanwhile, MCTS scales roughly linearly with iteration count, requiring 2.0 seconds at 200 iterations, 5.9 seconds at 700, and 15.7 seconds at 2000. A key observation is that MCTS-700, the tournament's strongest agent, spends approximately 13 times more time

per move than Minimax-7 (5.9 seconds versus 0.44 seconds). When considering per-move computation, we see that Minimax derives far more value from its compute budget, with Minimax-7 winning 27% of games against MCTS-700 while using one-thirteenth the time per move, and Minimax-9 pushing MCTS-700 to a near-even 56-44 split while using approximately two-thirds of MCTS-700's per-move time. The per-move time for each tested variation of each agent can be seen in Figure 5 above.

### *3.6 RL Agent Learning Curve*

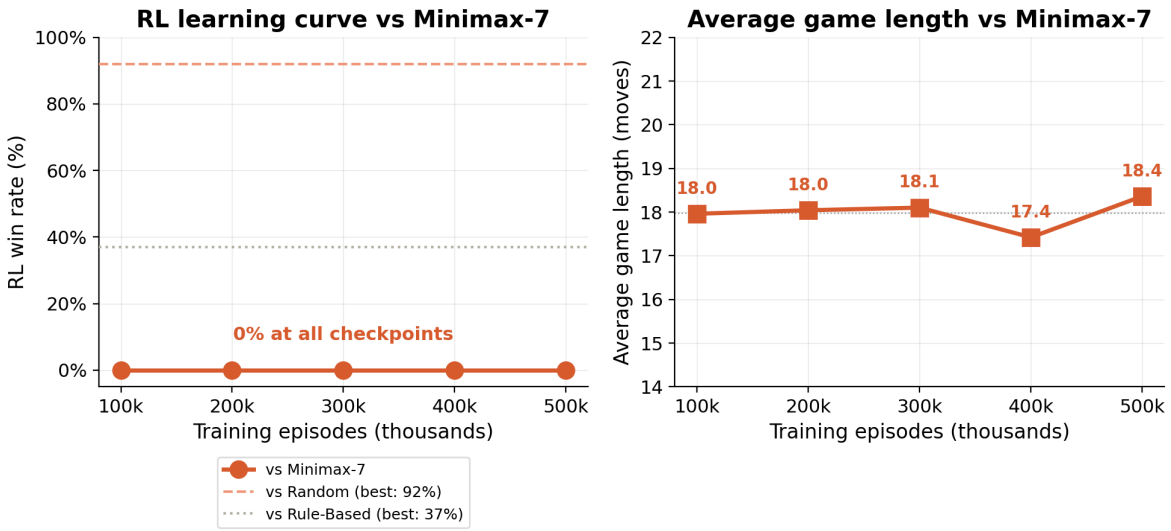


**Figure 6 - RL agent performance against various agents**

To assess whether the RL agent's poor tournament performance was a function of insufficient training, we evaluated checkpoints at 100,000, 200,000, 300,000, 400,000, and 500,000 training episodes against Minimax-7, playing 100 games at each checkpoint. As seen in Figure 6 above, the RL agent did not win a single game at any checkpoint, with increased training not leading to improved performance. The average game length remained approximately constant at 17.4 to 18.4 moves across all checkpoints, indicating that the agent was not learning to survive longer against Minimax even as training progressed. This flat

learning curve suggests that the pure self-play training regime, while sufficient to learn basic positional play (as evidenced by the 92% win rate against the random agent), fails to develop the multi-layer tactical reasoning required to compete with even shallow search-based agents. The RL agent never sees positions arising from strong adversarial play during training, so it never learns how to defend against the precise tactical sequences that Minimax and MCTS employ.

## 4. Conclusions

Our tournament produced a dominance hierarchy of MCTS > Minimax > RL, which partially refutes our initial hypothesis that Minimax would be the strongest agent. While our prediction that the RL agent would rank below the search-based agents was confirmed decisively, the relationship between MCTS and Minimax was the opposite of what the literature on branching factor and search traps suggested. MCTS-700 defeated Minimax-7 in 60% of 130 games, and this advantage held across both first-player and second-player positions, indicating a stable and systematic performance advantage rather than a chance result.

### *4.1 Why MCTS Outperformed Minimax*

The MCTS agent's advantage over Minimax likely stems from the quality of our MCTS implementation's heuristic enhancements rather than a fundamental model advantage. The MCTS agent uses smart rollouts that immediately play winning moves and block opponent threats, tree reuse that preserves search effort across consecutive moves, and tactical pre-checks that avoid wasting simulations on obvious moves. These enhancements effectively give MCTS the one-ply tactical awareness that Baier and Winands (2015) identified as the key weakness of pure MCTS in Connect 4, while retaining MCTS's ability to explore diverse strategic sequences that a fixed-depth Minimax may miss. Moreover, at 700

iterations per move, the MCTS agent spends approximately 5.9 seconds per move compared to Minimax-7's 0.44 seconds, a 13-fold compute difference that may account for much of the win rate difference. When Minimax is allowed to search deeper, to depth 9 at 3.9 seconds per move, the gap narrows to a 56–44 MCTS advantage, suggesting that with equalized compute budgets the two paradigms would be much closer to equal performance. This result is consistent with the broader finding of Baier and Winands (2018) that branching factor is crucial in determining the most suitable approach, meaning Connect 4's branching factor of seven is low enough for Minimax to be competitive, but not so low that MCTS's sampling-based approach is categorically at a disadvantage.

### *4.2 The RL Agent's Failure Mode*

The RL agent's performance was the most decisive result of the tournament, winning 0% of games against every Minimax and MCTS configuration tested, across all training checkpoints from 100,000 to 500,000 episodes. This outcome persisted even against Minimax at depth 3, the shallowest configuration, which takes the same per-move time as the RL agent itself. The flat learning curve, with no improvement across 400,000 additional training episodes, indicates that pure self-play training without any form of search hit a hard ceiling well below the level required to compete with explicit search methods in Connect 4.

The root cause is a training distribution mismatch. During self-play, the RL agent only encounters positions that arise from its own (far from perfect) play. It never sees the precise tactical sequences that Minimax and MCTS exploit, because those sequences require looking several steps ahead to construct. The tactical override mechanism, which forces immediate wins and blocks during training, addresses one-ply tactics but does not help the network learn to defend against two-ply or deeper threats. As a result, the network's value estimates are

calibrated for self-play positions, not for positions arising from strong adversarial play. The 37% win rate against the rule-based agent confirms that the RL agent learned some positional understanding, reaching 92% against the random agent, but failed to consistently acquire even the simple win-and-block heuristic that the rule-based agent implements. This suggests that pure self-play, without exposure to adversarial positions beyond the agent's own skill level, produces a network that is well-calibrated for its training distribution but ineffective when confronted with stronger, search-based opponents.

### *4.3 Compute Efficiency*

The compute cost analysis reveals a fundamental tradeoff between the three agent approaches. The RL agent is by far the fastest at inference time, at 0.004 seconds per move, but this speed is rendered meaningless by its inability to translate fast pattern matching into competent play. Minimax is highly compute-efficient in the sense that it derives substantial playing strength from reasonable per-move budgets. For example, Minimax-7 at 0.44 seconds per move outperforms the RL agent completely and loses to MCTS-700, which spends 13 times more computation. MCTS is the most compute-hungry approach, requiring seconds per move even at moderate iteration counts, but it converts that computation into the strongest play. This tradeoff suggests that in time-constrained settings, such as real-time gameplay with strict per-move limits, Minimax may be the more practical choice despite MCTS's higher ceiling.

### *4.4 Engineering Observations*

The development process revealed several engineering challenges relevant to the design of game-playing agents. The most impactful was the discovery that using Python’s collections.deque as a replay buffer causes sampling throughput to degrade linearly with

buffer size, due to O(n) random access. Replacing the deque with pre-allocated NumPy arrays and a circular write pointer restored O(1) sampling and was the single most important performance fix in the RL training pipeline. The vectorization of the game engine, replacing a sequential Python loop with a batched NumPy implementation processing 512 environments simultaneously, improved throughput by approximately two orders of magnitude, from 2 episodes per second to 93–165 episodes per second. The learning rate scheduler was initially misconfigured, with the cosine annealing period set relative to the total number of episodes rather than the actual number of optimizer steps, causing the learning rate to remain effectively flat throughout training. These issues highlight the gap between algorithmic correctness and practical performance that is characteristic of deep RL systems.

### *4.5 Future Work*

A natural next step for this project is combining the strengths of the RL and MCTS agents into a single hybrid. During development, we explored the idea of an AlphaZero-style agent (Silver et al., 2018) that would use the trained RL network inside MCTS. The policy head guides which branches to explore first via the PUCT selection formula, and the value head evaluates leaf positions directly instead of running rollouts to a terminal state. This would address the RL agent's core weakness, its lack of any lookahead at inference time, while making MCTS more efficient by replacing slow heuristic rollouts with fast neural network evaluations. A team member built a preliminary version of this hybrid (hybrid_mcts_agent.py), which plugs the RL value network into the MCTS loop in place of rollouts, though it was not included in the tournament evaluation. A full implementation would also incorporate the policy head as a search prior and retrain the network through the AlphaZero self-play loop, where MCTS-guided games produce richer training targets than the win/loss outcomes used in our pure self-play approach. Beyond the hybrid, other

promising directions include adding transposition tables to the Minimax agent to enable deeper search within the same time budget, and training the RL agent for longer or against progressively stronger opponents.

## 5. Works Cited


Allen, J. D. (1989). A note on the computer solution of Connect-Four. In L. Levy & D. Beal (Eds.), *Heuristic Programming in Artificial Intelligence: The First Computer Olympiad*. Ellis Horwood.

Allis, V. (1988). A Knowledge-Based Approach of Connect-Four: The Game Is Solved: White Wins. ICGA Journal, 11(4), 165–165. https://doi.org/10.3233/ICG-1988-11410

Nasa, R., Didwania, R., Maji, S., & Kumar, V. (2018). Alpha-beta pruning in mini-max algorithm–an optimized approach for a connect-4 game. *Int. Res. J. Eng. Technol*, *5*(4), 1637-1641. https://www.academia.edu/download/56816350/IRJET-V5I4366.pdf

Baier, H., & Winands, M. H. M. (2015). MCTS-Minimax Hybrids. IEEE Transactions on Computational Intelligence and AI in Games., 7(2), Article 6942254. https://doi.org/10.1109/TCIAIG.2014.2366555

Baier, H., & Winands, M. H. M. (2018). MCTS-Minimax Hybrids with State Evaluations. The Journal of Artificial Intelligence Research, 62, 193–231. https://doi.org/10.1613/jair.1.11208

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial Intelligence, 6(4), 293–326. https://doi.org/10.1016/0004-3702(75)90019-3

Kocsis, L., & Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. In J. Fürnkranz, M. Spiliopoulou, & T. Scheffer (Eds.), Lecture notes in computer science (Vol. 4212, pp. 282–293). Springer Berlin Heidelberg. https://doi.org/10.1007/11871842_29

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science (American Association for the Advancement of Science), 362(6419), 1140–1144. https://doi.org/10.1126/science.aar6404